PPI Curug Simulator Training — Full Documentation

> *Generated on: 2026-03-23 15.49.36*

Dokumen ini menggabungkan seluruh dokumentasi di folder docs/.

Dokumentasi — PPI Curug Simulator Training

Dokumentasi ini menjelaskan project **dari nol**: cara setup, routing (Next.js App Router), API routes, autentikasi + RBAC, Prisma + database, hingga panduan penggunaan per role.

Daftar Dokumen

1. **Quickstart & Setup Lokal**

- Lihat: `docs/01-quickstart.md`

2. **Routing (App Router) & Struktur Folder**

- Lihat: `docs/02-routing.md`

3. **Auth, Session, Email OTP, RBAC**

- Lihat: `docs/03-auth-rbac.md`

4. **Database & Prisma (MongoDB)**

- Lihat: `docs/04-database-prisma.md`

5. **API Reference (ringkas tapi menyeluruh)**

- Lihat: `docs/05-api-reference.md`

6. **Panduan Penggunaan (User/Admin/Finance/Instructor)**

- Lihat: `docs/06-usage-guide.md`

7. **Operasional (Uploads, SMTP, Cron Reminder, Branding)**

- Lihat: `docs/07-ops-branding.md`

Konvensi Respon API

Hampir semua endpoint API mengembalikan bentuk JSON yang konsisten:

- Sukses:

```
{ "ok": true, "data": { "...": "..." } }
```

- Gagal:

```
{ "ok": false, "error": { "message": "...", "details": "...optional..." } }
```

Helper-nya ada di `src/lib/http.ts`.

---

01 — Quickstart & Setup Lokal

1) Prasyarat

- Node.js (LTS disarankan)
- NPM
- Akses database MongoDB (contoh: MongoDB Atlas)

2) Install Dependency

Di root project `ppi-curug-simulator-training`:

```
npm install
```

Catatan: repo memiliki script `postinstall` yang otomatis menjalankan `prisma generate`.

3) Konfigurasi Environment

Buat file `.env` di root project. Template ada di `.env.example`.

Minimal yang wajib:

- `DATABASE_URL`
- `JWT_SECRET`

Opsional (tapi direkomendasikan):

- `UPLOAD_DIR` (default `uploads`)
- `APP_URL` (dibutuhkan untuk link reset password)
- `APP_BASE_URL` (dibutuhkan untuk tombol CTA di email template)

Untuk email/OTP (production disarankan pakai SMTP):

- `MAIL_TRANSPORT = smtp | ethereal | log`
- `SMTP_HOST`, `SMTP_PORT`, `SMTP_FROM`
- `SMTP_USER`, `SMTP_PASS` (jika SMTP butuh auth)

4) Inisialisasi Database (Prisma)

Project ini pakai Prisma dengan provider **MongoDB**.

Perintah penting:

```
npm run prisma:migrate
npm run db:seed
```

Penjelasan:

- `prisma:migrate` menjalankan `prisma db push` (meng-apply schema ke MongoDB).
- `db:seed` menjalankan `prisma/seed.ts` untuk membuat akun seed + data awal.

Jika perlu generate client manual:

```
npm run prisma:generate
```

5) Jalankan Development Server

```
npm run dev
```

Buka:

- http://localhost:3000

6) Akun Seed (Development)

Secara default (hasil seed):

- `admin / admin123`
- `finance / finance123`
- `instructor / instructor123`

Role `USER` dibuat via registrasi.

7) Build & Run Production

Build:

```
npm run build
```

Run:

```
npm start
```

8) Troubleshooting Cepat

Prisma error EPERM di Windows

Jika muncul error semacam: `EPERM: operation not permitted, rename ... query_engine-windows.dll.node.tmp`

Biasanya ada proses `node.exe` yang masih mengunci file engine Prisma. Solusi:

1. Stop semua proses `npm run dev` / `next dev`.
2. Pastikan tidak ada node yang masih running.
3. Jalankan ulang:

```
npm run prisma:generate
```

Email/OTP di dev tidak terkirim

Jika SMTP belum dikonfigurasi, sistem akan fallback ke **log** (console server) di mode non-production. Lihat output terminal untuk kode OTP/link.

02 — Routing (Next.js App Router) & Struktur Folder

Project ini menggunakan **Next.js App Router**.

A) Struktur Folder Utama

- `src/app/` → semua **route** (UI pages, layouts) + **route handlers** (API)
- `src/lib/` → util server (prisma, session, rbac, email, audit, dll)
- `prisma/` → schema + seed
- `public/` → static assets (logo, ikon, dll)
- `uploads/` → default lokasi file upload (jika `UPLOAD_DIR` tidak diubah)

B) Cara Next.js Membangun Route

1) Page routes (UI)

Dalam App Router:

- Folder + `page.tsx` menjadi route.
- Folder + `layout.tsx` menjadi layout wrapper untuk subtree.
- Segment dinamis memakai `[param]`, misalnya `user/certificates/[id]/page.tsx` menghasilkan route:
  - `/user/certificates/:id`

Contoh:

- `src/app/page.tsx` → `/`
- `src/app/login/page.tsx` → `/login`

2) API routes (Route Handlers)

Dalam App Router:

- Folder `src/app/api/**/route.ts` membentuk endpoint `/api/**`.
- Method ditentukan oleh export function: `GET`, `POST`, `PUT`, `PATCH`, `DELETE`.

Contoh:

- `src/app/api/auth/login/route.ts` → `POST /api/auth/login`
- `src/app/api/documents/upload/route.ts` → `POST /api/documents/upload`

3) Static file routes

- File di `public/` tersedia langsung dari root URL.
  - `public/logoppic/logoppic.7a5aa04c.png` → `/logoppic/logoppic.7a5aa04c.png`

Khusus App Router icons:

- `src/app/icon.png` → `/icon.png` (dipakai sebagai icon/tab/fav di banyak browser).

C) Peta Route UI (Ringkasan)

> *Catatan: sebagian halaman adalah area role tertentu dan dijaga oleh middleware.*

Public

- `/` → landing/home (hero slideshow, info, schedule preview)
- `/login`, `/register`
- `/forgot-password`, `/reset-password`
- `/verify-email`

General (butuh login)

- `/dashboard`
- `/account` (menu akun)
- `/account/profile` (re-export ke halaman profil user)

USER

- `/user/dashboard`
- `/user/profile`
- `/user/documents`
- `/user/booking/new`
- `/user/bookings/[id]`
- `/user/certificates/[id]`

ADMIN

- `/admin/dashboard`
- `/admin/dashboard/report`
- `/admin/landing`
- `/admin/logs`
- `/admin/bookings`
- `/admin/schedule`
- `/admin/users`
- `/admin/verifications`

FINANCE

- `/finance/dashboard`
- `/finance/dashboard/report`
- `/finance/payments`
- `/finance/legal`

INSTRUCTOR

- `/instructor/dashboard`
- `/instructor/schedule`
- `/instructor/users`
- `/instructor/logbook`

D) Middleware & Guard Routing

File: `src/middleware.ts`

Fungsi utama middleware:

1. Menentukan route yang **public** dan **protected**.
2. Jika tidak login → redirect ke `/login?next=...`.
3. Memvalidasi JWT dari cookie session.

4. Memaksa verifikasi email sebelum akses aplikasi (kecuali endpoint OTP + logout).
5. Guard area role:
    - `/admin/*` hanya untuk role `ADMIN`
    - `/finance/*` hanya untuk role `FINANCE`
    - `/instructor/*` hanya untuk role `INSTRUCTOR`
    - `/user/*` hanya untuk role `USER`

Cookie yang relevan

- `ppic_session` → JWT session (httpOnly)
- `ppic_device_id` → device id (httpOnly) untuk audit log / pelacakan perangkat

E) Responsiveness (UI Routing)

UI menggunakan Tailwind dan komponen di `src/app/*`. Beberapa route UI berupa "page wrapper" yang re-export. Contoh:

- `src/app/account/profile/page.tsx` mengekspor default dari `src/app/user/profile/page.tsx`.

Tujuan pola ini: URL `/account/profile` tetap ada, tapi implementasi halaman profil tetap satu sumber.

F) Detail `route.ts`: ctx.params di Dynamic Segment

Pada route handler dengan segment dinamis, Next.js App Router memberikan params lewat argumen kedua. Di repo ini Anda akan melihat pola seperti:

```
export async function POST(_req: NextRequest, ctx: { params: Promise<{ id: string }> }) {
  const { id } = await ctx.params;
  // ...
}
```

Kenapa `params` bertipe `Promise`?

- Ini mengikuti pola App Router terbaru untuk memastikan params bisa di-resolve secara async.

G) Cara Menambah Route Baru (Praktis)

1) Menambah halaman UI

Misal ingin menambah halaman `/admin/reports`:

1. Buat folder: `src/app/admin/reports/`
2. Buat file: `src/app/admin/reports/page.tsx`

Minimal isi:

```
export default function ReportsPage() {
  return <div>Reports</div>;
}
```

Jika halaman ini harus admin-only:

- Pastikan prefix path-nya `/admin/...` (middleware akan menolak role selain ADMIN).

2) Menambah endpoint API

Misal ingin menambah endpoint `GET /api/admin/reports/summary` :

1. Buat folder: `src/app/api/admin/reports/summary/`
2. Buat file: `route.ts`

Contoh pola implementasi yang konsisten di repo:

```
import { jsonOk, jsonError } from "@/lib/http";
import { requireRole } from "@/lib/rbac";

export async function GET() {
  const { session, response } = await requireRole(["ADMIN"]);
  if (!session) return response;
  try {
    return jsonOk({ message: "ok" });
  } catch (e) {
    return jsonError("Server error", 500);
  }
}
```

3) Menambah asset publik

Untuk menambah file statis yang bisa diakses langsung URL:

- Taruh di `public/` .

Contoh:

- `public/brand/logo.png` → bisa diakses lewat `/brand/logo.png` .

---

03 — Auth, Session, Email OTP, RBAC

Dokumen ini menjelaskan alur login, session cookie, verifikasi email OTP, reset password, dan RBAC.

A) Session: Cookie + JWT

File utama:

- `src/lib/session.ts`

1) Cookie

Nama cookie session:

- `ppic_session`

Cookie diset sebagai:

- `httpOnly: true`
- `sameSite: "lax"`
- `secure: true` saat production

- `maxAge: 8 jam`

2) JWT Payload

Tipe payload session:

```
type SessionPayload = {
  userId: string;
  username: string;
  role: "USER" | "ADMIN" | "FINANCE" | "INSTRUCTOR";
  emailVerified: boolean;
};
```

JWT ditandatangani dengan secret:

- `JWT_SECRET`

JWT expiry:

- 8 jam

B) Middleware Guard

File: `src/middleware.ts`

Yang dilakukan middleware:

1. Mengecualikan route public + static assets.
2. Jika route protected dan tidak ada cookie `ppic_session` → redirect ke `/login?next=...`.
3. Jika ada token dan `JWT_SECRET` tersedia → verifikasi token.
4. Jika `emailVerified=false` → paksa redirect ke `/verify-email` (kecuali endpoint OTP dan logout).
5. Guard role untuk `/admin`, `/finance`, `/instructor`, `/user`.

C) RBAC untuk API

File: `src/lib/rbac.ts`

Helper:

- `requireSession()` → memastikan login
- `requireRole([roles...])` → memastikan role sesuai

Pola umum di route handler:

```
const { session, response } = await requireRole(["ADMIN"]);
if (!session) return response;
// ...lanjut logic
```

Jika tidak login → 401. Jika role tidak sesuai → 403.

D) Login

Endpoint:

- `POST /api/auth/login`

File: `src/app/api/auth/login/route.ts`

Ringkasan alur:

1. Validasi input (Zod): `identifier` dan `password`.
2. Cari user berdasarkan:
    - username
    - email
    - email di profile
3. Verifikasi password (bcrypt).
4. Update `lastLoginAt`, `lastLoginIp`, `lastLoginUserAgent` (best-effort).
5. Tulis audit log `auth.login` (best-effort) termasuk `ip`, `userAgent`, `deviceId`.
6. Sign JWT + set cookie session.

E) Logout

Umumnya dilakukan dengan menghapus cookie `ppic_session`. Lihat endpoint logout di `src/app/api/auth/logout/route.ts`.

F) Registrasi

Endpoint:

- `POST /api/auth/register`

Umumnya membuat user role `USER`, menyimpan `passwordHash`, lalu mengarahkan user untuk melengkapi profil/dokumen.

G) Verifikasi Email (OTP)

Konsep:

- OTP 6 digit.
- TTL 10 menit.
- Maks percobaan 3.
- Cooldown request OTP 60 detik.

File OTP core:

- `src/lib/emailVerification.ts`

Secret hashing OTP:

- `OTP_SECRET` (jika ada)
- fallback: `JWT_SECRET`
- fallback terakhir (dev): string default

Endpoint:

- `POST /api/auth/otp/request` → minta OTP
- `POST /api/auth/otp/verify` → verifikasi OTP

Middleware memaksa user yang `emailVerified=false` untuk menyelesaikan ini sebelum akses fitur lain.

H) Reset Password

Terdapat 2 mekanisme yang dipakai:

1) Reset password via token link

File:

- `src/lib/passwordReset.ts`

Env:

- `APP_URL` dipakai untuk base URL link reset.

Alur:

1. User meminta reset.
2. Sistem membuat token random, simpan hash di DB dengan TTL.
3. Kirim link `/reset-password?token=...`.
4. Token dikonsumsi sekali.

2) Reset password via OTP

File:

- `src/lib/passwordResetOtp.ts`

Konsep mirip verifikasi email OTP (secret bisa `OTP_SECRET`).

I) Device ID untuk Audit Log

Karena web app tidak dapat mengambil MAC address client secara realistis, project ini memakai pendekatan **Device ID**:

- Middleware memastikan cookie `ppic_device_id` selalu ada.
- Audit log menyimpan `deviceId` ke `metadata.deviceId`.

File:

- `src/middleware.ts` (set cookie)
- `src/lib/audit.ts` (ambil deviceId dari header/cookie)
- `src/lib/audit.ts` + `src/lib/audit.ts` (write log)

Catatan:

- Device ID bukan identitas perangkat yang "pasti", tetapi cukup untuk korelasi aktivitas pada browser/perangkat tertentu.

J) Detail: Sumber `emailVerified`

Di database, status verifikasi email disimpan sebagai:

- `User.emailVerifiedAt` (DateTime | null)

Saat login (`POST /api/auth/login`), aplikasi menghitung:

- `emailVerified = Boolean(emailVerifiedAt)`

Nilai `emailVerified` ini dimasukkan ke JWT session payload.

Di `src/lib/session.ts` ada kompatibilitas:

- Token yang diterbitkan sebelum field `emailVerified` ada akan dianggap `true`.

K) Catatan Penting: `JWT_SECRET` wajib di Production

Middleware akan mencoba memverifikasi cookie session memakai `JWT_SECRET`. Jika `JWT_SECRET` tidak diset, middleware saat ini melakukan *best-effort* dan membiarkan request lewat.

Implikasi:

- Di production, **wajib** set `JWT_SECRET` agar guard bekerja benar.

L) Mengakses Session di Server Component

Contoh penggunaan session di layout:

- `src/app/layout.tsx` memanggil `getSessionFromCookies()` lalu meneruskan data ke `HeaderNav`.

Pola ini penting karena:

- `layout.tsx` adalah server component, sehingga bisa membaca cookie `httpOnly`.
- Komponen client (mis. `HeaderNav.tsx`) menerima session sebagai props untuk render UI.

---

04 — Database & Prisma (MongoDB)

A) Prisma Overview

Folder:

- `prisma/schema.prisma` → schema utama (MongoDB)
- `prisma/seed.ts` → seed data

Client Prisma:

- `src/lib/prisma.ts` membuat singleton `PrismaClient` (aman untuk Next dev hot-reload).

B) Menjalankan Perubahan Schema

Perintah yang dipakai project:

```
npm run prisma:migrate
```

Script tersebut menjalankan:

- `prisma db push`

Artinya:

- Prisma akan menyinkronkan schema ke MongoDB (tanpa migration file seperti SQL).

C) Seed Data

Perintah:

```
npm run db:seed
```

Seed biasanya membuat:

- akun `admin`, `finance`, `instructor`
- setting awal yang diperlukan

D) Model Penting

Berikut gambaran model inti (ringkas):

1) User

- Menyimpan `username`, `passwordHash`, `role`
- `emailVerifiedAt` untuk status verifikasi email
- `lastLoginAt`, `lastLoginIp`, `lastLoginUserAgent`

2) Profile

- Biodata user
- `registrationType`: personal / company
- `status`: `PENDING/APPROVED/REJECTED`
- relasi ke `Document[]`

3) Document

- File dokumen user: `LICENSE`, `MEDICAL`, `ID`, dll
- `storagePath` menunjuk file di disk (`UPLOAD_DIR`)
- status verifikasi

4) Booking

- Berisi pemesanan simulator
- memiliki `status` yang cukup panjang (lihat enum `BookingStatus`)

5) ScheduleSlot

- Slot jadwal simulator, status: `AVAILABLE/LOCKED/BOOKED`

6) Payment & LegalDocument

- Payment status: `UNPAID/PAID/VALIDATED/REJECTED`
- Dokumen legal: `PKS`, `BERITA_ACARA` dengan status `DRAFT/ISSUED`

7) Certificate

- Sertifikat booking yang bisa diakses dan memuat QR code

8) AuditLog

- Tabel log aktivitas
- Field: `action`, `targetType`, `targetId`, `metadata`, `ip`, `userAgent`, `createdAt`
- `metadata.deviceId` dipakai untuk korelasi perangkat

9) Notification

- Notifikasi per user

- `readAt` untuk status terbaca

E) Enum Status (yang sering dipakai UI)

- `Role` : USER/ADMIN/FINANCE/INSTRUCTOR
- `VerificationStatus` : PENDING/APPROVED/REJECTED
- `BookingStatus` : DRAFT → WAIT_ADMIN_VERIFICATION → WAIT_FINANCE_DOCS → WAIT_PAYMENT → PAYMENT_VALIDATION → CONFIRMED → COMPLETED (dan CANCELLED)
- `PaymentStatus` : UNPAID/PAID/VALIDATED/REJECTED

F) Prisma Studio

Untuk melihat data:

```
npm run prisma:studio
```

G) Catatan MongoDB Unique Index & Sparse

Karena beberapa field email bersifat opsional, pastikan index unik tidak mengunci nilai `null` di banyak dokumen. Project memiliki script helper:

```
npm run db:fix-mongo-indexes
```

Gunakan ini jika terjadi konflik unique index saat field opsional.

---

05 — API Reference

Dokumen ini merangkum endpoint utama. Semua endpoint berada di `src/app/api/**` .

A) Konvensi

- Base path: `/api/...`
- Response JSON konsisten via `src/lib/http.ts` .
- Role guard:
  - Banyak endpoint memakai `requireRole([...])` dari `src/lib/rbac.ts` .

B) Auth

Folder: `src/app/api/auth/`

- `POST /api/auth/register`
  - Membuat user baru (umumnya role USER)
- `POST /api/auth/login`
  - Login, set cookie `ppic_session`
- `POST /api/auth/logout`
  - Hapus cookie `ppic_session`

- `POST /api/auth/otp/request`
  - Kirim OTP verifikasi email
- `POST /api/auth/otp/verify`
  - Verifikasi OTP
- `POST /api/auth/password-reset/request`
  - Minta reset password (token/link atau OTP tergantung flow)
- `POST /api/auth/password-reset/verify-otp`
  - Verifikasi OTP reset password
- `POST /api/auth/password-reset/confirm`
  - Konfirmasi reset password

C) Me / Profile

- `GET /api/me`
  - Mengembalikan info user login + info profil ringkas
- `GET/POST /api/profile`
  - Mengambil/mengubah data profil

D) Documents

- `POST /api/documents/upload`
  - Upload dokumen user (file + metadata)
- `GET /api/documents/[id]/download`
  - Download dokumen
- `GET /api/documents`
  - List dokumen (bergantung implementasi route)

E) Simulators & Slots

- `GET /api/public/simulators`
- `GET /api/public/slots`

Endpoint public ini dipakai di landing/schedule preview.

Admin/Staff biasanya punya endpoint khusus untuk kelola slot:

- `GET/POST /api/admin/slots`
- `PATCH/DELETE /api/admin/slots/[id]`
- `POST /api/admin/slots/bulk`

F) Bookings

User:

- `GET /api/bookings` (list)
- `GET /api/bookings/[id]` (detail)
- `POST /api/bookings/[id]/submit` (submit draft)
- `POST /api/bookings/[id]/select-slot` (memilih slot)

Admin/Staff:

- `POST /api/admin/bookings/[id]/approve`
- `POST /api/admin/bookings/[id]/certificate`
- `POST /api/admin/bookings/staff-book`

Instructor:

- `POST /api/instructor/bookings/staff-book`

G) Payments

- `GET /api/payments`
- `POST /api/payments/[id]/mark-paid`

Finance:

- `GET /api/finance/payments`
- `GET /api/finance/payments/[id]/proof` (lihat bukti bayar)
- `POST /api/finance/payments/[id]/validate` (validasi / reject)

H) Legal Documents (Finance)

- `GET /api/finance/legal-documents`
- `POST /api/finance/legal-documents/[id]/upload`
- `GET /api/finance/legal-documents/[id]/download`

I) Notifications

- `GET /api/notifications`
- `POST /api/notifications/[id]/read`

J) Audit Logs (Admin)

- `GET /api/admin/logs`
- `GET /api/admin/logs/export`

Audit log menyimpan:

- `ip`
- `userAgent`
- `metadata.deviceId` (device id cookie)

K) Landing Config & Images

Public landing config:

- `GET /api/public/landing`

Admin mengelola landing:

- `GET/POST /api/admin/landing`
- `POST /api/admin/landing-images/upload`

Asset landing image disajikan dari:

- `GET /api/public/landing-images/[name]`

L) Cron Reminders

Endpoint:

- `GET/POST /api/public/cron/reminders/run`

Header auth (jika `REMINDER_CRON_SECRET` diset):

- `x-cron-secret: <secret>` atau
- `Authorization: Bearer <secret>`

Query:

- `dryRun=1` untuk simulasi
- `limit=<n>` untuk batasi jumlah proses

---

06 — Panduan Penggunaan (End User)

Dokumen ini berfokus ke cara memakai aplikasi dari sisi pengguna (UI).

A) Alur Utama User (ROLE: USER)

1) Registrasi

1. Buka `/register`.
2. Isi username + password (dan email jika diminta).
3. Setelah sukses, login lewat `/login`.

2) Verifikasi Email (OTP)

Jika setelah login diarahkan ke `/verify-email`:

1. Pastikan email sudah diisi (di profil).
2. Klik **Kirim OTP**.
3. Masukkan kode OTP yang diterima.

Catatan dev:

- Jika SMTP tidak dikonfigurasi, OTP akan muncul di console server.

3) Lengkapi Profil

Halaman:

- `/user/profile` (atau `/account/profile`)

Isi:

- biodata dasar
- email, alamat, dll

4) Upload Dokumen

Halaman:

- `/user/documents`

Upload dokumen sesuai tipe:

- LICENSE
- MEDICAL
- ID
- dll

Dokumen akan berstatus:

- PENDING → menunggu verifikasi

5) Buat Booking

Halaman:

- `/user/booking/new`

Langkah:

1. Pilih simulator (Airbus/Boeing) dan training.
2. Pilih skema lease (WET/DRY).
3. Simpan → booking akan dibuat dengan status awal `DRAFT`.

6) Submit Booking

Dari detail booking (`/user/bookings/[id]`):

1. Klik submit.
2. Status berubah menjadi `WAIT_ADMIN_VERIFICATION`.
3. Sistem akan membuat notifikasi + (opsional) email.

Catatan:

- Untuk lease `WET`, dokumen wajib biasanya harus lengkap (LICENSE, MEDICAL, ID) kecuali email user masuk allowlist `DOCS_BYPASS_EMAILS`.

7) Menunggu Verifikasi Admin

Admin memverifikasi profil/dokumen dan menyetujui booking.

8) Menunggu Dokumen Finance

Setelah admin approve, status biasanya berpindah ke tahap finance. Finance menerbitkan dokumen legal / instruksi pembayaran.

9) Upload Bukti Pembayaran

User mengunggah bukti pembayaran lewat fitur pembayaran (biasanya dari detail booking atau menu pembayaran).

10) Validasi Pembayaran

Finance memvalidasi:

- Jika VALIDATED → lanjut penjadwalan/konfirmasi.
- Jika REJECTED → user diminta upload ulang bukti bayar.

11) Pilih Slot Jadwal

Jika booking sudah masuk tahap penjadwalan:

1. User memilih slot yang tersedia.
2. Status slot berubah menjadi `BOOKED`.

12) Sertifikat

Jika booking selesai dan admin menerbitkan sertifikat:

- User bisa membuka `/user/certificates/[id]`.
- Sertifikat biasanya berisi QR code untuk validasi.

B) Alur Admin (ROLE: ADMIN)

Halaman utama:

- `/admin/dashboard`

Aktivitas umum:

1. Verifikasi profil user: `/admin/verifications`
2. Verifikasi dokumen user: `/admin/verifications`
3. Kelola booking: `/admin/bookings`
    - approve booking
    - terbitkan sertifikat
4. Kelola slot jadwal: `/admin/schedule`
5. Kelola user: `/admin/users`
6. Audit logs: `/admin/logs`
    - lihat aktivitas (IP, user-agent, deviceId)
    - export CSV
7. Kelola landing/home: `/admin/landing`

C) Alur Finance (ROLE: FINANCE)

Halaman utama:

- `/finance/dashboard`

Aktivitas umum:

1. Lihat pending booking: (sesuai menu finance)
2. Terbitkan dokumen legal:
    - `/finance/legal`
    - upload/preview/download dokumen
3. Kelola pembayaran:
    - `/finance/payments`
    - lihat bukti bayar
    - validasi / reject
4. Export report:

- `/finance/dashboard/report`

D) Alur Instructor (ROLE: INSTRUCTOR)

Halaman utama:

- `/instructor/dashboard`

Aktivitas umum:

1. Jadwal: `/instructor/schedule`
2. Isi logbook untuk wet leased: `/instructor/logbook`
3. Akses data user tertentu (jika ada): `/instructor/users`

E) Notifikasi

Widget notifikasi:

- `src/app/NotificationsWidget.tsx`

API:

- `GET /api/notifications`
- `POST /api/notifications/[id]/read`

Notifikasi dibuat pada event tertentu (contoh submit booking).

F) Tips Pemakaian

- Jika icon/tab browser belum berubah, lakukan hard refresh atau buka incognito (favicon sering cache).
- Jika file upload gagal, cek permission folder `uploads/` (atau `UPLOAD_DIR`) dan pastikan server dapat menulis file.

---

07 — Operasional (Uploads, SMTP, Cron, Branding)

A) Uploads (File Storage)

Project menyimpan file upload ke disk (bukan ke database).

Env:

- `UPLOAD_DIR` (default: `uploads`)

Pola implementasi:

- API menerima `multipart/form-data`
- File disimpan sebagai `uploads/<generated-name>`
- Database menyimpan metadata (mimeType, fileName, storagePath)

Endpoint terkait (contoh):

- `POST /api/documents/upload`
- `GET /api/documents/[id]/download`
- `POST /api/finance/legal-documents/[id]/upload`

- `GET /api/finance/legal-documents/[id]/download`

Catatan production:

- Pastikan folder upload berada di volume persistent.
- Pastikan permission write sesuai.

B) SMTP / Email Delivery

File:

- `src/lib/mailer.ts`

Mode:

- `MAIL_TRANSPORT=smtp` (default)
- `MAIL_TRANSPORT=ethereal` (hanya non-production)
- `MAIL_TRANSPORT=log` (selalu log, tidak mengirim)

Konfigurasi minimum SMTP:

- `SMTP_HOST`
- `SMTP_PORT`
- `SMTP_FROM`

Jika butuh auth:

- `SMTP_USER`
- `SMTP_PASS`

Base URL untuk link email:

- `APP_URL` (reset password link)
- `APP_BASE_URL` (CTA di template email)

C) Cron Reminders

Endpoint:

- `/api/public/cron/reminders/run`

Env:

- `REMINDER_CRON_SECRET`

Behavior:

- Jika `REMINDER_CRON_SECRET` kosong (dev) → endpoint bisa dipanggil tanpa auth.
- Jika production dan secret tidak ada → endpoint menolak dan mengembalikan error 500 (server not configured).

Cara memanggil:

```
curl -H "x-cron-secret: <secret>" "https://<host>/api/public/cron/reminders/run?dryRun=1&limit=10"
```

D) Device ID & Audit Log

Cookie device id:

- `ppic_device_id` (httpOnly)

Dibuat di middleware (`src/middleware.ts`) agar setiap browser/perangkat punya id stabil. Audit log akan merge `deviceId` ke `metadata.deviceId` saat menulis log (`src/lib/audit.ts`).

E) Branding Assets (Logo & Icon Tab)

Logo header saat ini:

- `public/logoppic/logoppic.7a5aa04c.png`

Icon/tab (App Router):

- `src/app/icon.png` → otomatis menjadi `/icon.png`

Catatan:

- Browser sangat agresif cache favicon/icon. Jika terasa belum berubah, hard refresh atau incognito.

F) UI Theme Ringkas

- Brand color utama: `#05164d`
- Global "square corners" ada di `src/app/globals.css` dengan pengecualian untuk avatar (`data-keep-rounded="true"`).

G) Scripts Operasional

Lihat `package.json` untuk script database dan maintenance:

- `npm run db:fix-mongo-indexes`
- `npm run db:clear-slots`
- `npm run db:check-conflicts`
- dst.